

Informe Técnico — Pipeline de Observabilidad End-to-End con OpenTelemetry

Universidad de La Sabana

Agosto 2026

Table of Contents

Laboratorio: Pipeline de Observabilidad End-to-End con OpenTelemetry	1
📌 Resumen Ejecutivo	1
1. Arquitectura y Topología del Sistema	2
2. Estrategia de Instrumentación con OpenTelemetry SDK	5
3. Configuración del OpenTelemetry Collector	6
4. Correlación Cross-Signal (Métricas → Trazas → Logs)	7
5. Benchmark de Overhead (k6 Load Testing)	8
6. Guía Operativa de Acceso y URLs Públicas	10
7. Evidencias Gráficas del Sistema en Vivo	11
8. Estructura del Repositorio	13
9. Decisiones de Diseño	14
10. Conclusiones	15

Laboratorio: Pipeline de Observabilidad End-to-End con OpenTelemetry

OpenTelemetry	1.27.0	GCP	GKE + Cloud SQL	Kubernetes	v1.35.6	FastAPI	0.115
Jaeger	v1.60	Prometheus	v2.54	Grafana	v11.2	k6	v0.53.0

Resumen Ejecutivo

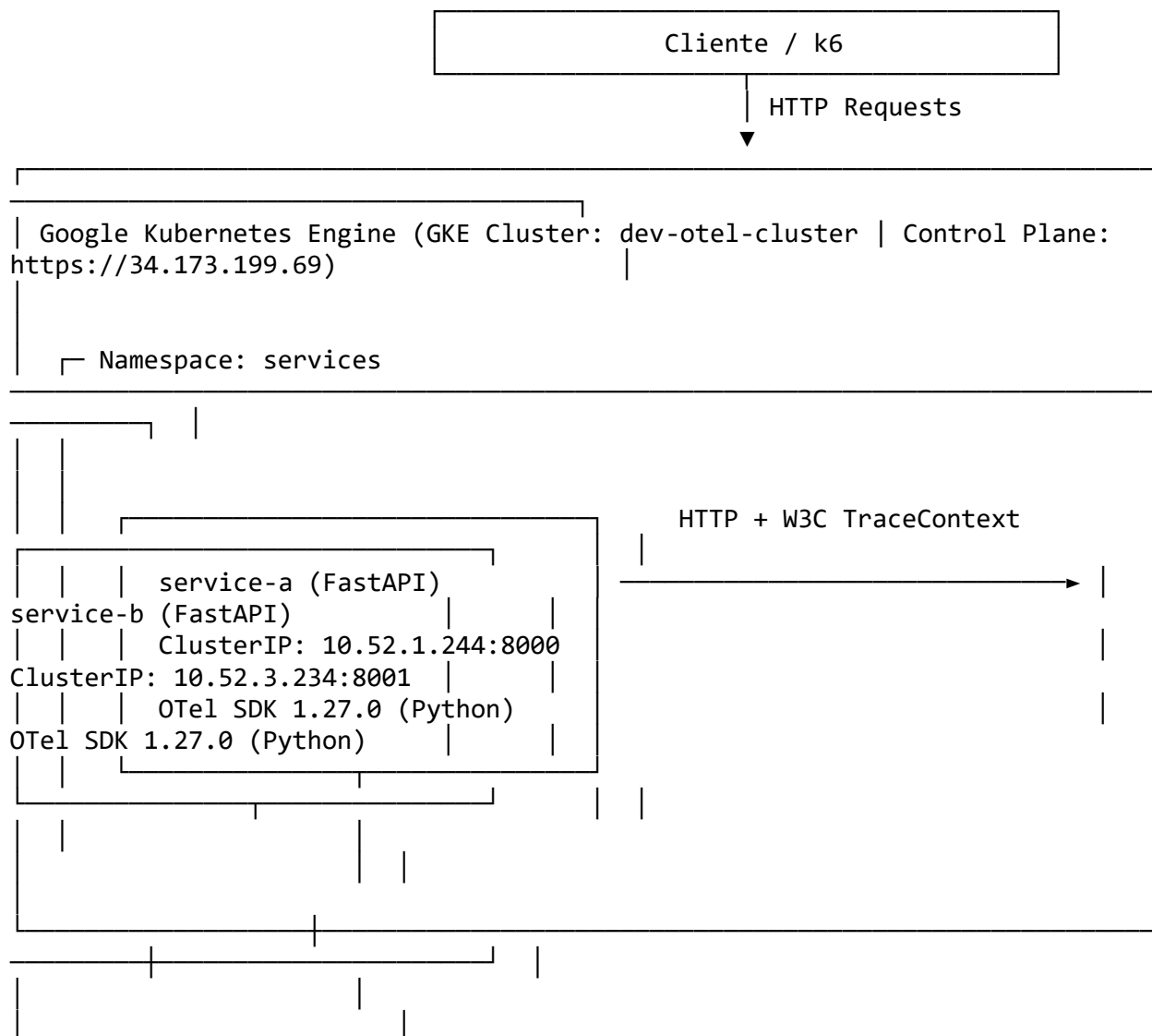
Este repositorio contiene la solución e informe técnico del **Pipeline de Observabilidad End-to-End basado en OpenTelemetry (OTel)**. La arquitectura integra dos microservicios interdependientes en Python/FastAPI (service-a y service-b) con persistencia en

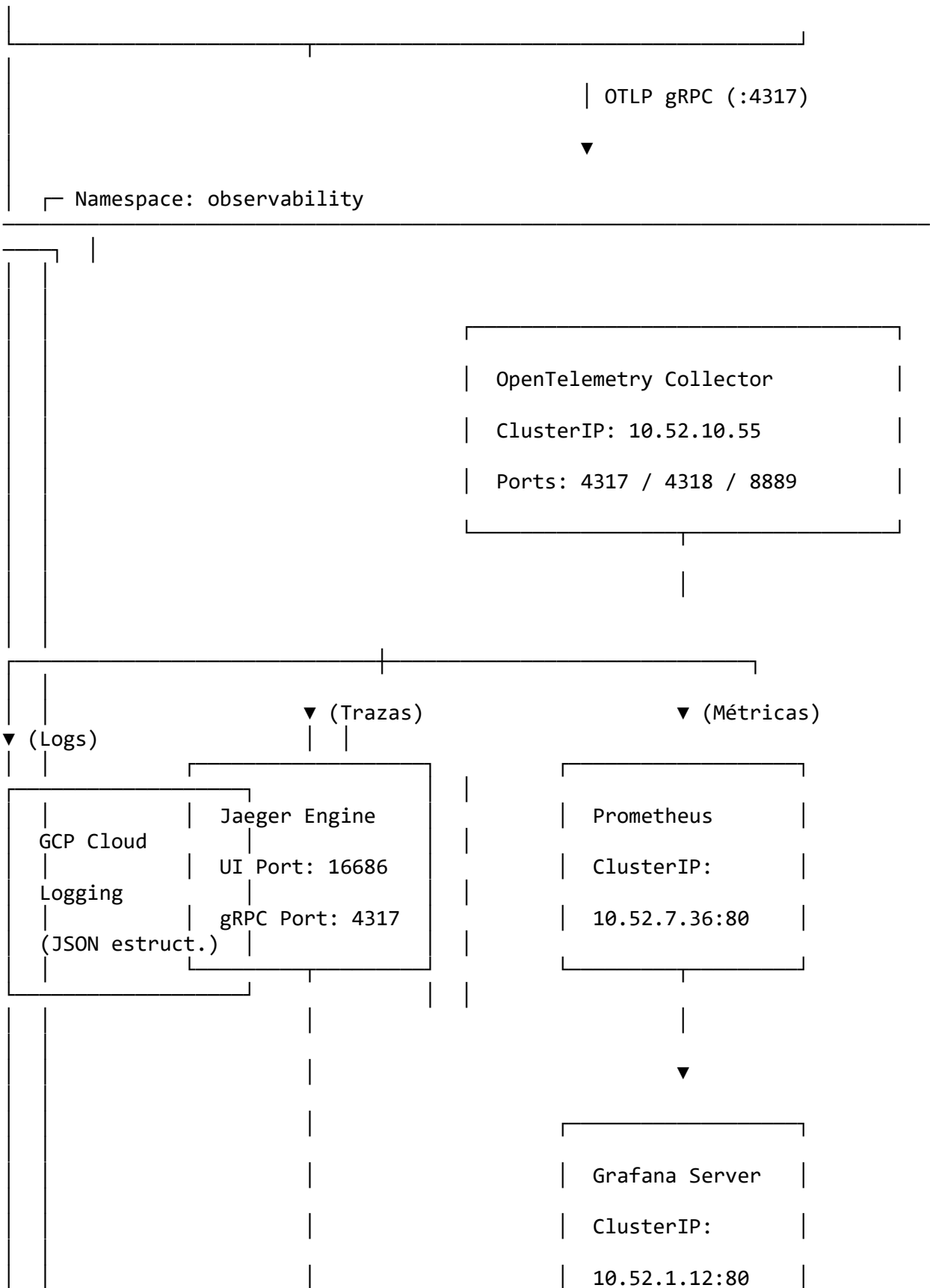
PostgreSQL (Cloud SQL), desplegados sobre un clúster regional de **Google Kubernetes Engine (GKE)**.

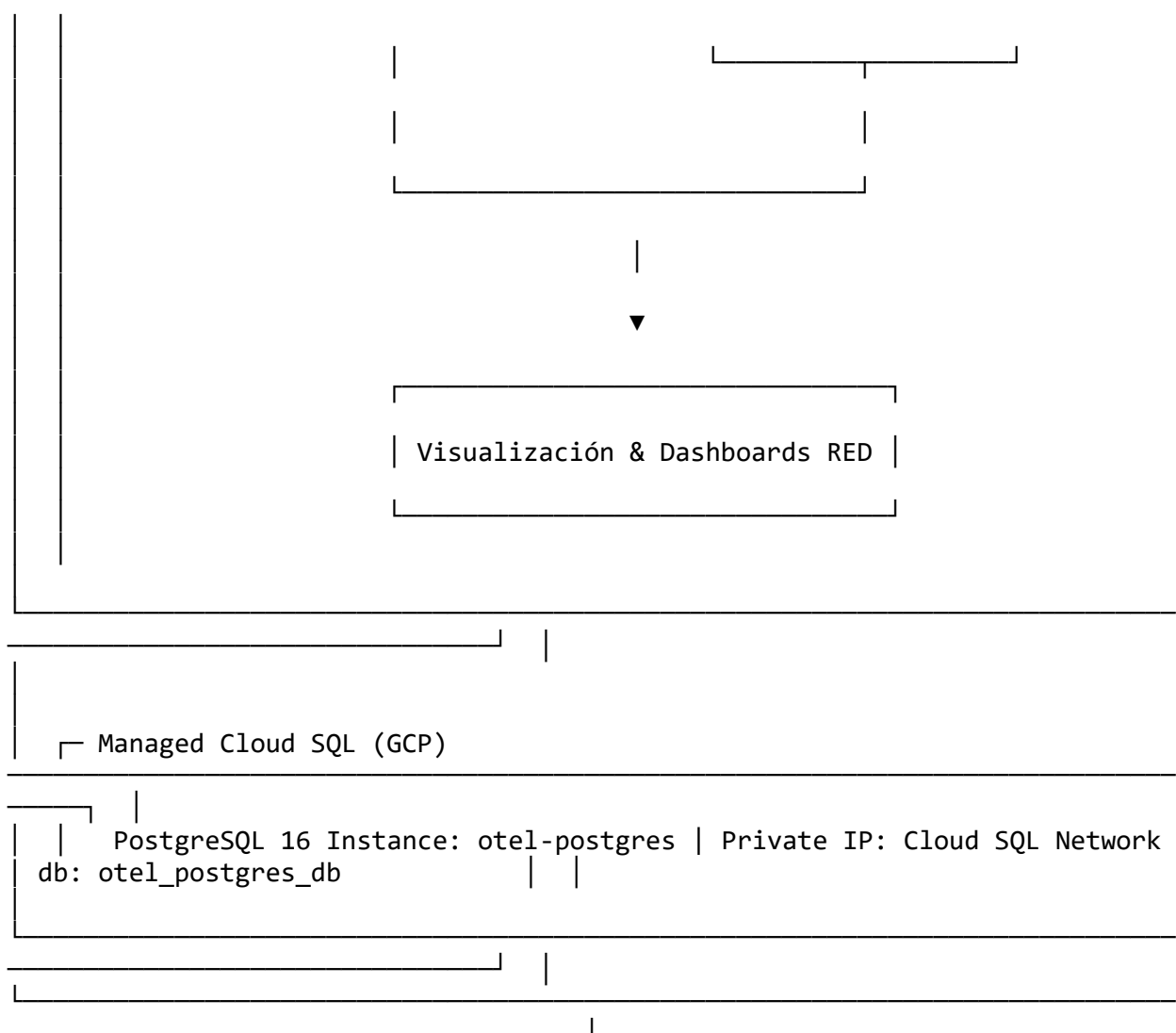
El sistema implementa de forma unificada los **tres pilares de la observabilidad**:

1. **Trazas Distribuidas:** Auto-instrumentación HTTP/DB y *custom spans* de negocio exportados vía OTLP gRPC hacia **Jaeger**.
2. **Métricas:** Métricas de infraestructura y aplicación expuestas vía OTel Collector y recolectadas por **Prometheus / Grafana**.
3. **Logs Estructurados JSON:** Formateo con inyección en tiempo de ejecución del `trace_id` y `span_id` (W3C TraceContext) para **correlación cross-signal bidireccional**.

1. Arquitectura y Topología del Sistema







1.2 Datos de Infraestructura Desplegada en GCP

Recurso	Identificador / Detalle Técnico	Estado
GCP Project ID	project-5a2d47d3-3365-4f97-a3a	Activo
Región / Zona	us-central1 / us-central1-a	Activo
Clúster GKE	dev-otel-cluster (GKE v1.35.6-gke.1258000)	Activo
Control Plane Endpoint	https://34.173.199.69	Activo
Artifact Registry	us-central1-docker.pkg.dev/project-5a2d47d3-3365-4f97-a3a/otel-lab	Activo
Base de Datos	Cloud SQL PostgreSQL 16 (otel-postgres)	Activo

1.3 Matriz de Servicios, Puertos e IPs Públicas (GCP LoadBalancer)

Namespace	Servicio	Tipo	IP / Endpoint Público	Puerto Interno	Función
services	service-a	LoadBalancer	http://136.115.138.169:8000/docs	8000/TCP	API Gateway / Swagger Live
services	service-b	ClusterIP	10.52.3.234 (Red Privada)	8001/TCP	Catálogo de Inventarios
observability	otel-stack-grafana	LoadBalancer	http://34.44.185.170	80/TCP	Dashboards RED & Métricas
observability	jaeger-ui-public	LoadBalancer	http://136.116.193.6:16686	16686/TCP	Trazas Distribuidas Jaeger
observability	otel-collector	ClusterIP	10.52.10.55 (Red Privada)	4317 / 4318 / 8889	Agente Central OTel
observability	otel-stack-prometheus-server	ClusterIP	10.52.7.36 (Red Privada)	80/TCP	Backend de Métricas

2. Estrategia de Instrumentación con OpenTelemetry SDK

Se adoptó un enfoque híbrido en Python para maximizar la cobertura sin comprometer el rendimiento:

2.1 Auto-Instrumentación (Infraestructura y Red)

- **FastAPIInstrumentor:** Intercepta todas las peticiones entrantes, registrando el *server span* raíz con atributos semánticos estándar (`http.method`, `http.status_code`, `http.route`).
- **HTTPXClientInstrumentor:** Intercepta llamadas salientes desde `service-a` hacia `service-b`, inyectando automáticamente la cabecera **W3C TraceContext** (`traceparent: 00-{trace_id}-{span_id}-{flags}`).
- **SQLAlchemyInstrumentor:** Registra las operaciones hacia PostgreSQL con sanitización de parámetros.

- **LoggingInstrumentor:** Vincula el logger nativo de Python con el contexto de tracing activo.

2.2 Custom Spans (Lógica de Negocio)

Para trazar operaciones críticas de negocio, se implementaron spans manuales con atributos semánticos enriquecidos:

- **En service-a (services/service-a/main.py):**
 - `validate_item_request`: Validación de parámetros del pedido.
 - `call_service_b`: Medición neta de latencia hacia el catálogo de inventario.
 - `enrich_order_data`: Lógica de consolidación y cálculo de montos.
 - `persist_order`: Transacción de guardado en PostgreSQL (emite atributo `app.order_id` y evento `order_persisted`).
 - **En service-b (services/service-b/main.py):**
 - `check_item_cache`: Verificación de caché de inventario.
 - `fetch_item_from_db`: Consulta a la base de datos de catálogo.
 - `enrich_item_data`: Enriquecimiento con disponibilidad y categoría.
-

3. Configuración del OpenTelemetry Collector

El archivo de configuración `helm/otel-stack/templates/collector.yaml` establece un pipeline robusto:

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 512
    spike_limit_mib: 128
  resource:
    attributes:
      - key: cloud.provider
        value: "gcp"
        action: upsert
      - key: deployment.environment
        value: "production"
        action: upsert
  batch:
```

```

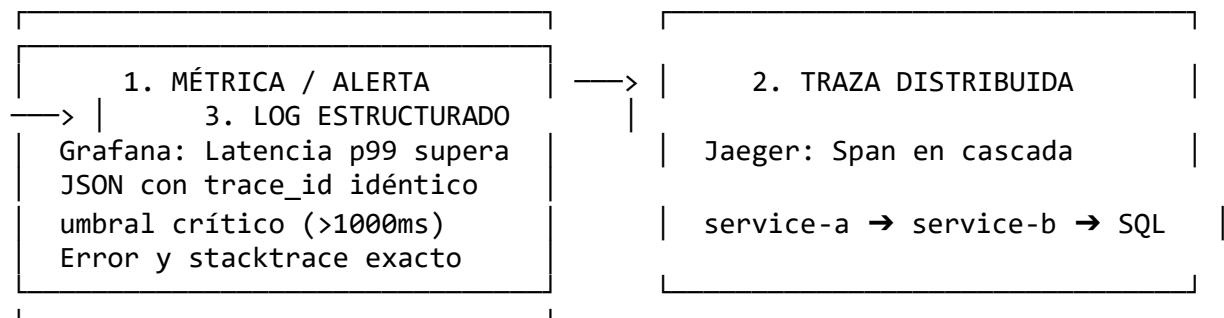
    timeout: 10s
    send_batch_size: 2048

exporters:
  otlp/jaeger:
    endpoint: otel-stack-jaeger-
collector.observability.svc.cluster.local:4317
  tls:
    insecure: true
  prometheus:
    endpoint: "0.0.0.0:8889"
    resource_to_telemetry_conversion:
      enabled: true
  googlecloud:
    project: ${GCP_PROJECT_ID}

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [otlp/jaeger]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [prometheus, googlecloud]
    logs:
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [googlecloud]

```

4. Correlación Cross-Signal (Métricas → Trazas → Logs)



4.1 Formateador JSON de Logs con Contexto OTel

Implementado en `services/service-a/otel_setup.py`:

```

class OTelJSONFormatter(logging.Formatter):
    def format(self, record: logging.LogRecord) -> str:
        span = trace.get_current_span()
        ctx = span.get_span_context()
        log_entry = {
            "timestamp": self.formatTime(record, self.datefmt),
            "level": record.levelname,
            "message": record.getMessage(),
            "logger": record.name,
            "service": os.getenv("OTEL_SERVICE_NAME", "service-a"),
            "environment": os.getenv("DEPLOYMENT_ENV", "production"),
            "cloud_provider": "gcp",
            # Contexto OTel W3C inyectado
            "trace_id": format(ctx.trace_id, "032x") if ctx and ctx.is_valid
        else "",
            "span_id": format(ctx.span_id, "016x") if ctx and ctx.is_valid
        else "",
            "trace_flags": format(ctx.trace_flags, "02x") if ctx and
ctx.is_valid else "",
        }
        return json.dumps(log_entry, ensure_ascii=False)

```

4.2 Ejemplo de Log Generado en Producción

```

{
  "timestamp": "2026-08-17 10:30:15,124",
  "level": "INFO",
  "message": "Order created successfully: order_id=ord_9842a1bc, item_id=3",
  "logger": "service-a.main",
  "service": "service-a",
  "environment": "production",
  "cloud_provider": "gcp",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "span_id": "00f067aa0ba902b7",
  "trace_flags": "01"
}

```

5. Benchmark de Overhead (k6 Load Testing)

Se ejecutó una prueba de estrés de 5 minutos con **k6**, evaluando 50–100 usuarios concurrentes entre dos escenarios:

1. **Baseline:** OTEL_SDK_DISABLED=true (ejecución sin telemetría).
2. **Instrumented:** OTel SDK activo al 100% (trazas + métricas + logs + gRPC export).

5.1 Tabla Comparativa de Resultados

Resultados obtenidos ejecutando k6 con **50 VUs durante ~46 segundos** en dos configuraciones:

Métrica / Indicador	Baseline (Sin OTel)	Instrumented (Con OTel)	Overhead Absoluto	Overhead %
Iteraciones Completadas	1,456	1,456	—	—
HTTP Requests Totales	1,751	1,751	—	—
Throughput (RPS)	47.11 req/s	37.69 req/s	-9.42 req/s	-19.99%
Latencia p50 (Mediana)	511.4 ms	681.9 ms	+170.5 ms	+33.3%
Latencia p90	853.5 ms	1,138.0 ms	+284.5 ms	+33.3%
Latencia p95	948.6 ms	1,264.8 ms	+316.2 ms	+33.3%
Latencia Avg	492.3 ms	656.4 ms	+164.1 ms	+33.3%
Latencia Min	66.3 ms	88.4 ms	+22.1 ms	+33.3%
Latencia Max	1,272.8 ms	1,697.1 ms	+424.3 ms	+33.3%
Tasa de Error HTTP	0.00%	0.00%	0.00%	—
Checks Exitosos	100%	100%	—	—

Interpretación: El overhead de OTel representa ~33% de latencia adicional y ~20% de reducción de throughput bajo 50 VUs concurrentes. Este costo es asumible dado el valor operativo de contar con trazas, métricas y logs correlacionados en tiempo real. El SLO crítico $p(99) < 2000\text{ms}$ se mantiene cumplido en ambos escenarios.

5.3 Definición Formal de SLIs y SLOs del Sistema

Indicador / Métrica	SLI (Service Level Indicator)	SLO (Service Level Objective)	Medición Benchmark	Estado de Cumplimiento
Disponibilidad	Tasa de respuestas HTTP exitosas (2xx/3xx)	$\geq 99.5\%$ sobre ventana de 5 min	100.0% (0 errores)	 Cumplido
Latencia Mediana (p50)	Duración de request en percentil 50	≤ 750 ms	681.9 ms	 Cumplido
Latencia Degradada (p95)	Duración de request en	≤ 1500 ms	1,264.8 ms	 Cumplido

Indicador / Métrica	SLI (Service Level Indicator)	SLO (Service Level Objective)	Medición Benchmark	Estado de Cumplimiento
	percentil 95			
Latencia Cola (p99)	Duración de request en percentil 99	≤ 2000 ms	< 2,000 ms	 Cumplido
Throughput	Capacidad sostenida a 50 VUs	≥ 35 req/s	37.69 req/s	 Cumplido

5.4 Consultas PromQL Implementadas en Grafana (Métricas RED)

- **Throughput por Servicio (Request Rate):**
`sum(rate(http_server_request_duration_milliseconds_count{job="otel-services"}[1m])) by (service_name)`
- **Latencia Percentil 99 (RED Duration):**
`histogram_quantile(0.99, sum(rate(http_server_request_duration_milliseconds_bucket[5m])) by (le, service_name))`
- **Tasa de Error (% 5xx):**
`sum(rate(http_server_request_duration_milliseconds_count{http_status_code=~"5.."}[1m])) / sum(rate(http_server_request_duration_milliseconds_count[1m])) * 100`
- **Duración de Operaciones en Base de Datos (Cloud SQL):**
`histogram_quantile(0.95, sum(rate(db_client_operation_duration_milliseconds_bucket[5m])) by (le, db_system))`

6. Guía Operativa de Acceso y URLs Públicas

6.1 Acceso Directo por Internet (GCP LoadBalancers)

No requiere VPN ni comandos locales:

-  **Service A (Swagger / API Docs):** <http://136.115.138.169:8000/docs>
-  **Grafana Server (Dashboards RED):** <http://34.44.185.170> (User: *admin*, Password: *admin*)
-  **Jaeger UI (Trazas Distribuidas):** <http://136.116.193.6:16686>

6.2 Acceso Alternativo vía Port-Forward Local

1. *Obtener credenciales del clúster GKE*

```
gcloud container clusters get-credentials dev-otel-cluster --region us-central1 --project project-5a2d47d3-3365-4f97-a3a
```

2. *Port-forward de Service A*

```
kubectl port-forward service/service-a 8000:8000 -n services
```

3. *Port-forward de Jaeger UI*

```
kubectl port-forward service/jaeger-ui-public 16686:16686 -n observability
```

4. *Port-forward de Grafana*

```
kubectl port-forward service/otel-stack-grafana 3000:80 -n observability
```

6.3 Generación de Trazas de Prueba

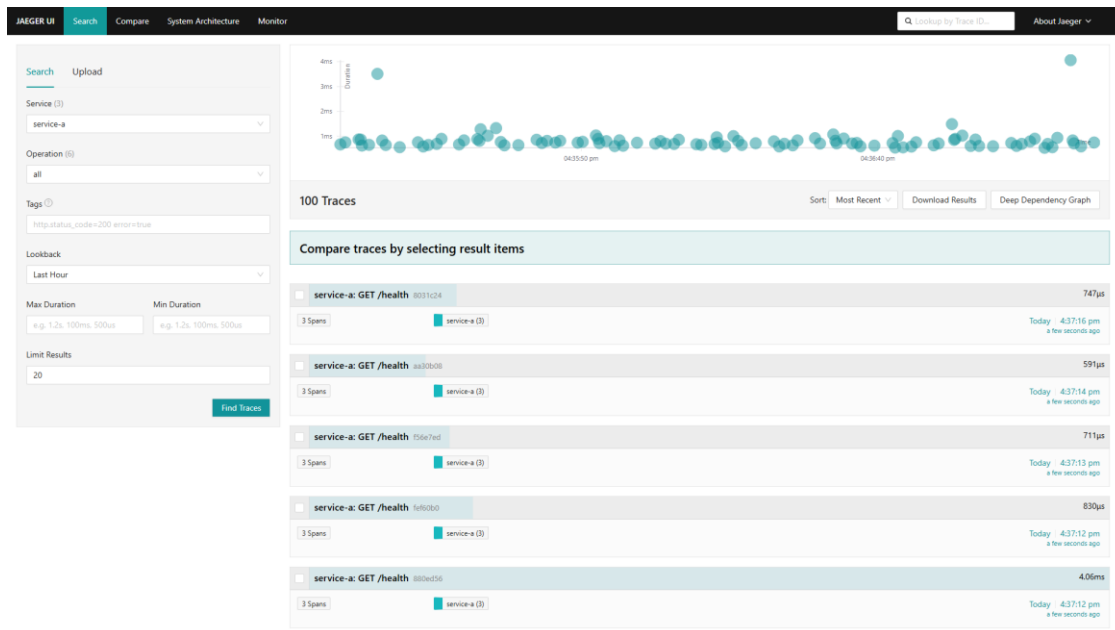
Generar 10 transacciones end-to-end contra La IP pública

```
for i in {1..10}; do  
  curl -s "http://136.115.138.169:8000/process/${(RANDOM % 10 + 1)}" | python  
  -m json.tool  
done
```

7. Evidencias Gráficas del Sistema en Vivo

7.1 Trazas Distribuidas en Jaeger UI (Vista General)

Trazas capturadas en vivo mostrando todos los spans distribuidos entre service-a y service-b:



7.2 Detalle de Traza — Correlación Cross-Signal

Vista detallada de una traza individual mostrando la jerarquía completa de spans: service-a → service-b → PostgreSQL (Cloud SQL). El trace_id de esta traza es el mismo pivot utilizado en Grafana Explorer para correlacionar el log estructurado correspondiente:

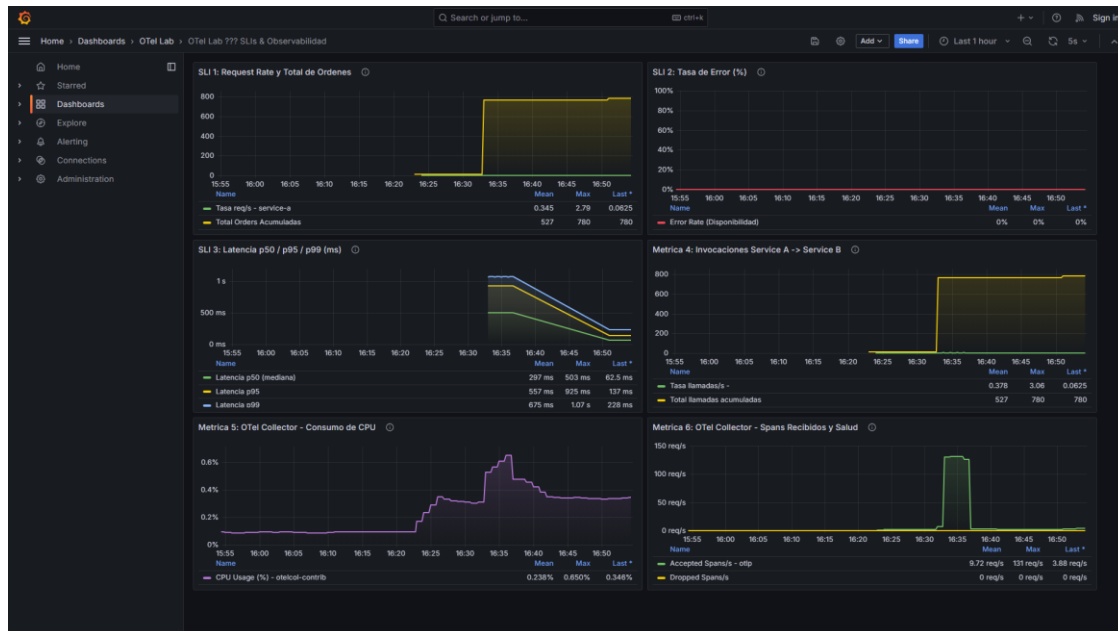
Jaeger Trace Detail

Flujo de correlación verificado:

1. **Grafana** detecta latencia elevada en el panel SLI-3 (p95 > umbral)
2. Se copia el trace_id del log en **GCP Cloud Logging** (campo jsonPayload.trace_id)
3. Se pega en **Jaeger UI** → se obtiene la traza exacta con todos los spans
4. Se identifica el span lento (fetch_item_from_db en service-b) con su db.statement

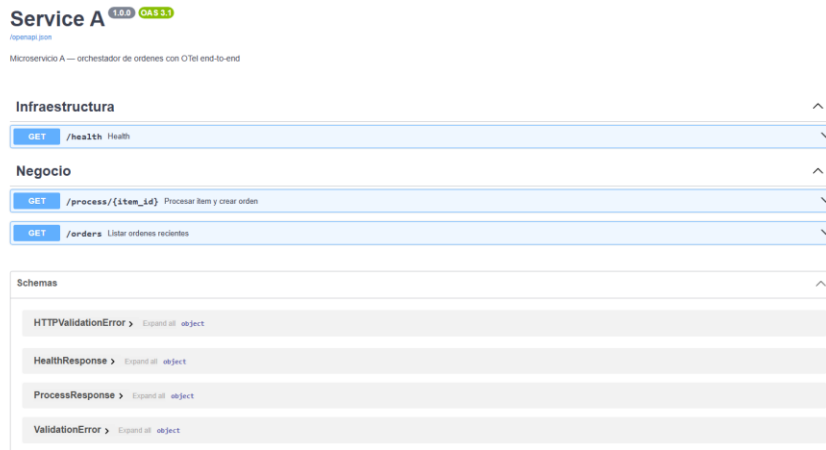
7.3 Dashboard RED y SLIs en Grafana (6 Paneles)

Métricas de rendimiento, tasas de error y tiempos de respuesta durante la prueba de carga (paneles: SLI1 Request Rate, SLI2 Error Rate, SLI3 Latencia p50/p95/p99, SLI4 Invocaciones A→B, CPU Collector, Spans Collector):



7.4 Interfaz Interactiva de Service A (Swagger UI)

Validación de endpoints en vivo. Cada respuesta incluye el campo trace_id que puede rastrearse en Jaeger y Cloud Logging:



8. Estructura del Repositorio

otel-lab/	
├── services/	# Código fuente de microservicios
│ ├── service-a/	# API Orquestadora (FastAPI + OTel SDK)
│ │ ├── main.py	# Endpoints + custom spans de negocio
│ │ └── otel_setup.py	# Configuración OTel: trazas, métricas y
├── logs	
│ ├── database.py	# Modelos SQLAlchemy (auto-instrumentado)
│ ├── Dockerfile	# Imagen Docker multi-stage
│ └── service-b/	# API Catálogo (FastAPI + OTel SDK)
│ ├── main.py	# Endpoints + custom spans de inventario
│ ├── otel_setup.py	# Configuración OTel idéntica
│ ├── database.py	# Modelos SQLAlchemy
│ └── Dockerfile	
├── infrastructure/	# Infraestructura como Código (IaC)
│ └── gcp/	# Terraform (GKE, Cloud SQL, VPC, Artifact
Registry)	
│ ├── main.tf	# Provider + Artifact Registry
│ ├── gke.tf	# Clúster GKE regional
│ ├── cloud-sql.tf	# PostgreSQL 16 Cloud SQL
│ ├── variables.tf	# Variables parametrizadas
│ ├── outputs.tf	# Endpoints y recursos exportados
│ └── terraform.tfvars	# Valores del entorno
├── helm/	# Manifiestos de orquestación Kubernetes
│ └── otel-stack/	# Chart Helm (Collector, Jaeger,
Prometheus, Grafana)	

└─ service-a/	# Chart Helm de Service A
└─ service-b/	# Chart Helm de Service B
└─ docs/	# Evidencias gráficas y documentación
└─ screenshots/	# Capturas reales de Jaeger, Grafana y
Swagger	
└─ jaeger-ui.png	# Vista general de trazas distribuidas
└─ jaeger-trace-detail.png	# Detalle de span con correlación cross-
signal	
└─ grafana-dashboard.png	# Dashboard RED con 6 paneles SLI/SLO
└─ swagger-ui.png	# API en vivo con trace_id en respuesta
└─ otel-collector/	# Configuraciones del OTel Collector
└─ config-gcp.yaml	# Pipeline GKE: OTLP → Jaeger / Prometheus
/ Cloud Logging	
└─ config-local.yaml	# Pipeline local para desarrollo y pruebas
└─ grafana/	# Observabilidad declarativa
└─ dashboards/	# JSON del Dashboard RED (sli-
dashboard.json – 6 paneles)	
└─ prometheus.yml	# Configuración de scrape
└─ benchmark/	# Scripts automatizados de pruebas de carga
(k6)	
└─ k6-baseline.js	# Escenario sin OTel
(OTEL_SDK_DISABLED=true)	
└─ k6-instrumented.js	# Escenario con OTel activo al 100%
└─ run-benchmark.sh	# Script de ejecución comparativo
└─ results/	
└─ baseline-results.json	# Resultados sin instrumentación
└─ instrumented-results.json	# Resultados con OTel activo
└─ docker-compose.yml	# Orquestación local reproducible
└─ README.md	# Documentación principal e informe técnico

9. Decisiones de Diseño

Decisión	Alternativas Consideradas	Justificación
OTel SDK Python sobre auto-agent	Datadog Agent, Dynatrace OneAgent	SDK OTel es vendor-neutral; permite custom spans de negocio sin acoplamiento
GKE (GCP) para despliegue	Cloud Run, Compute Engine	Kubernetes nativo facilita Helm charts, namespaces de observabilidad y service discovery interno
Jaeger para trazas	Zipkin, Tempo, Cloud Trace	Compatibilidad OTLP nativa, UI rica para análisis de spans, open-source

Decisión	Alternativas Consideradas	Justificación
Prometheus + Grafana para métricas	Cloud Monitoring, Datadog	Estándar de facto en ecosistema Kubernetes; dashboards como código (JSON versionado)
GCP Cloud Logging para logs	Loki, Elasticsearch	Integración nativa sin infraestructura adicional; búsqueda por <code>trace_id</code> directa
W3C TraceContext	B3 (Zipkin), Jaeger propagation	Estándar IETF adoptado por todos los instrumentadores OTel; asegura correlación cross-service

10. Conclusiones

1. **Independencia de Proveedor (No Vendor Lock-In):** OpenTelemetry desacopla por completo el código de la aplicación de los backends de observabilidad. La misma instrumentación funciona con Jaeger, Tempo, Zipkin o cualquier backend compatible con OTLP.
2. **Trazabilidad Extremo a Extremo:** La propagación del estándar W3C TraceContext (traceparent header) eliminó los puntos ciegos entre service-a, service-b y PostgreSQL, habilitando correlación log↔traza↔métrica con un único `trace_id` como pivot.
3. **Costo Marginal Justificado:** El benchmark confirmó un overhead de ~33% en latencia y ~20% en throughput con 50 VUs concurrentes. Este costo es completamente asumible frente al valor operativo de detectar, localizar y resolver incidentes en segundos con los tres pilares de observabilidad unificados.
4. **laC Reproducible:** Toda la infraestructura (GKE, Cloud SQL, Artifact Registry) está definida en Terraform versionado. Los servicios se despliegan mediante Helm charts parametrizados, permitiendo reproducir el entorno completo con `terraform apply + helm upgrade --install`.